

Project Documentation

SignLearn AI

Backend, artificial intelligence, and dataset contribution report

Prepared by Chinmayee Badiger
Team 4 | Infosys Springboard Internship 2026
Project guide: Shravya

September 2026

1. Project Information

- Project Title: SignLearn AI - Sign Language Learning and Assessment Platform
- Name: Chinmayee Badiger
- Team: Team 4, Infosys Springboard Internship 2026
- Project Guide: Shravya
- Role documented in member files: Backend and Dataset Lead
- Primary repositories reviewed: chinmayeebadiger/languagewebsite and springboardmentor15742s-oss/Team_4_Sign_Language_AI
- Template used: Final Document Overview.pdf, used as an outline and visual reference only

2. Abstract

This document records my full visible contribution to the SignLearn AI project. My work spans dataset research and setup guidance, a FastAPI AI gesture evaluation endpoint, MediaPipe landmark feature extraction, model training support, dataset metadata APIs, a dataset library user interface, backend routers for learning-platform features, project reports, API documentation, and a personal portfolio artifact. The project combines a React and Vite frontend with a FastAPI backend and an AI/ML layer based on 21 hand landmarks. The final implementation is demo-ready and educational: it proves the platform workflow while using synthetic or generated data in areas where full production datasets and persistence were not yet connected.

3. Introduction

3.1 Background

Sign language learning applications need more than static lesson pages. Learners need guided practice, immediate feedback, repeatable progress tracking, and clear access to learning content. SignLearn AI addresses this by combining lesson modules, camera-based gesture practice, assessment flows, learner analytics, and instructor-facing progress views.

3.2 Problem Statement

Traditional online resources can show signs but often cannot evaluate whether a learner performed a sign correctly. The project problem is to build a learning platform that can receive hand landmark data, classify or score a target sign, return coaching feedback, and organize the surrounding learning journey through courses, goals, history, and progress APIs.

3.3 Objectives

- Document and organize sign language datasets for model development.
- Build a backend endpoint that accepts 21 MediaPipe hand landmarks or a 63-value flat vector.
- Return predicted sign, accuracy percentage, correctness status, and correction tips for the frontend.
- Provide model training scripts and a feature-extraction path for landmark-based classification.
- Expose platform data through FastAPI routers for courses, progress, history, goals, leaderboard, notifications, and instructor analytics.
- Create a dataset library page and portfolio artifact to make the technical work visible and presentable.

4. Scope

4.1 In Scope

- My branches and local member folder: chinmayee-week1, chinmayee-week-2, chinmayee-week2-milestone2, chinmayee_milestone3, and 02_Chinmayee_Badiger.
- Final consolidated project files copied into backend, frontend, ml, docs, MEMBERS_WORK, and team_master.
- Backend routers, schemas, ML scripts, dataset documentation, dataset UI, branch history, and known limitations.

4.2 Out of Scope

- Large raw dataset downloads and private tokens, which are intentionally excluded by gitignore.
- A claim of production ML accuracy, because available scripts rely on synthetic landmarks or planned dataset extraction.
- Attributing teammate-authored documentation to me when the file itself names another author.
- Cloud deployment verification, because the local repo provides deployment instructions but no confirmed live production URL.

5. Existing System and Proposed Solution

5.1 Existing System

The existing learning approach before this project would be a static sign-language reference or video lesson flow. Such systems teach by showing examples, but they do not reliably score a learner's hand pose, do not return structured correction tips, and do not connect practice outcomes to progress, course enrollment, leaderboards, or instructor review.

5.2 Proposed Solution

The proposed solution is a full-stack learning platform. The browser captures or represents hand landmarks, sends them to a FastAPI backend, receives a structured evaluation response, and then displays practice feedback in the learning interface. Around that core loop, the backend exposes APIs for courses, learner progress, history, goals, notifications, leaderboards, and instructor analytics.

5.3 Key Features

- AI gesture evaluation API with simple and detailed response modes.
- Landmark normalization and 73-dimensional feature extraction in the Week 2 branch.
- Rule-based geometric evaluator in the final assembled backend.
- Dataset research, local storage rules, and preprocessing roadmap.
- RandomForest and optional KNN training path for landmark classification.
- Course catalog with 6 courses and 41 lessons.
- Learner progress, streak, weekly activity, history export, goals, notifications, leaderboard, and instructor statistics.
- Dataset library UI listing 12 public research datasets with filters, save state, links, and citation copy support.

6. Technology Stack

Layer	Technologies Used	My Use
Languages	Python, JavaScript, SQL, Markdown	Backend routers, ML scripts, frontend dataset page, documentation
Frontend	React, Vite, lucide-react, CSS	Dataset library page and personal portfolio app
Backend	FastAPI, Pydantic, Uvicorn	AI evaluation, dataset, courses, progress, history, goals, leaderboard, notification, instructor routers
Database	PostgreSQL schema, SQLAlchemy models in the final repo	Planned persistence for users, practice sessions, sign mastery, goals, courses, and enrollments
AI/ML	MediaPipe-style 21 landmarks, scikit-learn, RandomForest, KNN, NumPy, joblib	Feature extraction, classifier wrapper, training CLI, saved model artifact
Libraries / APIs	OpenCV, Pillow, MediaPipe optional paths, Kaggle dataset sources	Dataset loading and future landmark extraction from images/videos
Deployment	Docker, Docker Compose, Vite build, FastAPI service startup	Documented local and container-ready run paths; cloud deployment not verified

7. System Architecture

High-level flow used by the project:

Learner webcam or UI action

- > React frontend
- > FastAPI backend under /api
- > AI gesture evaluator and learning routers
- > Demo data or planned PostgreSQL storage
- > Structured feedback, progress, course, history, and analytics responses

The final backend registers feature routers for `ai_evaluate`, `progress`, `courses`, `instructor`, `goals`, `leaderboard`, `history`, and `notifications`. A dataset router also exists in the final files but is not included in the active backend/main.py router list.

7.1 System Modules

Module	Files Reviewed	Purpose
AI evaluation	backend/routers/ai_evaluate.py; backend/app/routers/ai_evaluate.py on branch	Receives landmarks, predicts or evaluates signs, returns accuracy and correction tips
ML services	backend/app/ml/*.py on chinmayee-week-2; ml/*.py in final folder	Feature extraction, classifier wrapper, synthetic bootstrapping, optional dataset-backed training
Dataset support	docs/Dataset_Integration_Guide.md;	Dataset research, storage rules, metadata

Module	Files Reviewed	Purpose
	dataset_docs/dataset_guide.md; backend/routers/dataset.py	endpoints, preprocessing plan
Learning APIs	courses.py, progress.py, history.py, goals.py	Course catalog, lesson access, mastery values, streaks, history export, learning goals
Engagement APIs	leaderboard.py, notifications.py, instructor.py	Rankings, in-app alerts, platform statistics, learner analytics
Presentation UI	DatasetLibraryPage.jsx; portfolio/src/main.jsx and styles.css	Research dataset browsing and personal contribution showcase

8. AI ML Implementation

8.1 Dataset

My dataset work begins in chinmayee-week1 with a Dataset Integration Guide and an implementation explanation. The guide compares Sign Language MNIST, ASL Alphabet, WLASL, and RWTH-PHOENIX, explains expected sizes and formats, and defines the raw and processed dataset folder layout. My final member work expands the visible catalog to 12 public datasets in the Dataset Library UI.

Dataset	Type / Scale	Project Role
Sign Language MNIST	28x28 grayscale CSV, 24 classes, about 34,627 images	Fast baseline and CI-friendly static alphabet experiments
ASL Alphabet	200x200 RGB images, 29 classes, about 87,000 images	Primary high-resolution static hand- shape source
WLASL	Dynamic ASL video, 100 to 2000 word subsets, 21,000+ clips	Future dynamic word recognition and sequence modeling
RWTH-PHOENIX	Continuous German Sign Language weather corpus	Advanced continuous recognition and translation reference, not primary ASL alphabet data
Dataset Library UI set	ASLLVD, OpenASL, ASL-Citizen, How2Sign, MS-ASL, AUTSL, NCSLGR, SignBD, CSL-Daily, INCLUDE, and others	Research catalog for evaluators and future model expansion

8.2 Data Processing

- Week 1 defines raw storage under datasets/raw and processed output under datasets/processed.
- The processing roadmap reshapes Sign Language MNIST to 28x28x1 arrays, normalizes pixels to 0-1, and preserves train/test structure.
- ASL Alphabet processing is planned as resize, normalization, stratified split, and careful augmentation because ASL orientation matters.
- WLASL processing is planned as frame sampling or MediaPipe landmark extraction over time with padded/truncated sequences.

- The Week 2 dataset loader reads MNIST CSVs, gathers ASL image paths, optionally loads ASL images through Pillow, and reports dataset availability for health checks.

8.3 Model Algorithm

The strongest branch implementation is chinmayee-week-2. It converts 21 MediaPipe keypoints into a model vector by centering landmarks at the wrist, scaling by palm size, flattening 63 normalized coordinates, and appending 10 pairwise fingertip distances for a 73-dimensional feature vector. The classifier wrapper loads a joblib artifact and supports RandomForest by default with KNN as an alternate model type.

The final assembled backend contains a simpler geometric classifier. It checks fingertip-to-wrist and knuckle distances, identifies extended fingers, distinguishes signs such as A, B, D, F, C, and HELLO, and returns coaching tips. This keeps the demo working even when a trained model is not wired into the active backend.

8.4 Training and Evaluation

- chinmayee-week-2 includes backend/app/ml/train_model.py for synthetic landmark bootstrapping and optional dataset-backed training.
- The branch writes backend/app/ml/artifacts/gesture_classifier.joblib and auto-bootstraps a prototype model if the artifact is missing.
- The final top-level ml/train_classifier.py trains a RandomForest pipeline on synthetic 63-feature samples for A-Z and common words.
- Reported evaluation is local script output based on synthetic data; no verified production benchmark against downloaded real datasets is present in the repo.

8.5 AI ML Workflow

MediaPipe landmarks

- > validate request with Pydantic schema
- > normalize landmarks and extract geometric features
- > run RandomForest/KNN classifier or rule-based evaluator
- > compare predicted sign to expected sign
- > return predicted_sign, accuracy_percentage, is_correct, and corrections

9. Database and API

9.1 Database

The final project includes a PostgreSQL schema for users, practice sessions, sign mastery, goals, courses, and enrollments. My active routers mostly return deterministic demo data instead of writing through this schema, which makes the frontend integration easy to demonstrate while leaving persistence as a future connection point.

Entity	Main Fields	Use
users	id, full_name, email, role, xp, streak_days, created_at	Learner and role identity
practice_sessions	id, user_id, sign_name, accuracy, passed, hand_count, created_at	Practice history and evaluation records

Entity	Main Fields	Use
sign_mastery	user_id, sign_name, mastery_pct	Per-sign progress tracking
goals	id, user_id, title, target_pct, current_pct, done	Learning goal state
courses and enrollments	course metadata plus user-course progress	Course catalog and learner enrollment

9.2 API Endpoints

Area	Endpoints	Contribution
AI evaluation	POST /api/ai/evaluate; POST /api/ai/evaluate/detailed; GET /api/ai/supported-signs; GET /api/ai/health	Gesture scoring, debug metadata, supported signs, module health
Datasets	GET /api/datasets/; GET /api/datasets/{dataset_key}	Dataset metadata API exists but is not mounted in final backend/main.py
Courses	GET /api/courses; GET /api/courses/enrolled/{learner_id}; GET /api/courses/{course_id}; GET /api/courses/{course_id}/lessons; POST /api/courses/{course_id}/enroll	6-course, 41-lesson catalog with enrollment response
Progress	GET /api/progress/{learner_id}; streak; weekly; summary; POST /api/progress/log	Mastery, streak, weekly activity, summary, and session-log response
History	GET /api/history/{learner_id}; stats; export	Paginated session history, aggregate stats, CSV export
Goals	GET /api/goals/{learner_id}; POST /api/goals; PATCH complete; DELETE goal	Learning-goal operations
Leaderboard	GET /api/leaderboard/weekly; all-time; rank	Weekly and all-time ranking data
Notifications and Instructor	GET/POST notification routes; GET/POST instructor routes	Unread alerts, mark-read response, learner stats, platform stats, course create response

10. User Interface

My visible UI work centers on DatasetLibraryPage.jsx and my personal portfolio. The final dataset page is a research catalog with 12 datasets, search, format filters, saved/starred state, external dataset links, citation copying, dataset statistics, and color-coded cards. The portfolio is a separate Vite React app presenting project cards, skills, milestone timeline, and contact/navigation sections.

Interface	Implemented Behavior	Why It Matters
Dataset Library	Search by name, language, format, tag, or organization; filter by format; save datasets; copy citation; open dataset links	Turns dataset research into an evaluator-friendly product page
Portfolio	Hero, work cards, skills, timeline, contact link, responsive styling	Presents my contribution as a professional artifact
Supported final app pages	Dashboard, practice, quiz, profile, courses, leaderboard, history, instructor dashboard	Backend routers provide the data shape these views need

11. GitHub and Version Control

11.1 Repository

- Final folder remote: <https://github.com/chinmayeebadiger/languagewebsite.git>
- Other project folder remote:
https://github.com/springboardmentor15742s-oss/Team_4_Sign_Language_AI.git
- Final folder main branch: main at commit 029704f, message 'explanations'
- Other folder active branch: chinmayee_milestone3 at commit 949425e

11.2 Repository Structure

Team_4_Sign_Language_AI_Final/
 backend/ FastAPI routers, schemas, auth, DB models and schema
 frontend/ React and Vite app pages and UI assets
 ml/ Dataset pipeline and classifier training script
 docs/ Architecture, API, dataset, deployment, milestone reports
 MEMBERS_WORK/ Archived member deliverables, including my folder
 team_master/ Consolidated team deliverables and duplicate master materials

11.3 Version Control

Branch / Commit	Date	Work Found
origin/chinmayee-week1 - f3e6f26	2026-08-04	Dataset Integration Guide, dataset folder layout, dataset processing explanation, gitignore placeholders
origin/chinmayee-week-2 - aa65036	2026-08-08	AI gesture endpoint, Pydantic schema, ML classifier service, correction generator, dataset loader, feature extractor, training CLI, saved model artifact
origin/chinmayee-week2-milestone2 - 4de54fe	2026-08-08	Milestone 2 completion updates, AI route/schema changes, dataset library page, layout/navigation additions, documentation
origin/chinmayee_milestone3 - a0e7e55	2026-08-28	Added 02_Chinmayee_Badiger folder with Milestone 3 and Milestone 4 deliverables, backend routers, ML files, docs, dataset UI, and portfolio

Branch / Commit	Date	Work Found
origin/chinmayee_milestone3 - 949 425e	2026-08-28	Removed the Milestone4 subfolder from that branch; final assembled folder still retains Milestone 4-style backend routers and docs
origin/chinmayee	2026-07-27	Empty branch aligned with mentor/main cleanup

12. Error Handling and Security

12.1 Error Handling

- The Week 2 AI endpoint validates landmark payloads and returns HTTP 422 when landmarks are missing or malformed.
- The Week 2 AI endpoint wraps model failures as HTTP 503 with a clear 'Gesture model unavailable' detail.
- The final geometric evaluator detects empty, short, or dummy landmark vectors and returns UNKNOWN with camera-position and lighting guidance.
- Course lookup raises HTTP 404 when a requested course ID is not found.
- Dataset lookup returns an error object when a dataset key is not recognized.

12.2 Security

- Raw and processed dataset folders are intentionally gitignored to avoid committing large licensed media or generated artifacts.
- Environment examples keep dataset and backend configuration out of hard-coded personal paths.
- The final backend enables CORS for demo use; production should restrict allowed origins.
- Authentication and profile files exist in the final project, but the active backend/main.py does not mount auth routes in its current router loop.
- The project documentation discusses JWT and role-based access, but the current demo routers mostly operate without enforced authorization.

13. Testing

The repository is structured for manual and smoke testing through Swagger UI, curl, local Vite frontend testing, and classifier training commands. The strongest explicit testing evidence is the Week 2 definition of done and run instructions, not a full automated test suite.

Test Area	How It Is Tested	Status
AI endpoint	Run backend, open /docs, submit 21 landmark objects or a 63-value landmarks_flat payload	Manual smoke-test path documented
Model training	python -m app.ml.train_model or python ml/train_classifier.py	Training path present; real-dataset evaluation not verified
Dataset availability	GET /api/ai/health in Week 2 branch reports dataset folder status	Implemented in branch service

Test Area	How It Is Tested	Status
Courses and progress APIs	Swagger/curl against mounted /api routers	Demo responses available
Frontend dataset page	Vite app rendering, search/filter/save/citation interactions	Implemented in React

14. Results

14.1 AI ML Results

- A working API response contract was created: predicted_sign, accuracy_percentage, is_correct, and corrections.
- The Week 2 branch supports 73-dimensional landmark features and a joblib-backed classifier wrapper.
- The final backend supports A-Z plus HELLO, THANK YOU, YES, NO, PLEASE, and SORRY in its supported-signs response.
- The final training script lists 40 sign classes and saves a model bundle to ml/model/sign_classifier.pkl.
- No production benchmark against downloaded real datasets is present; reported accuracy should be treated as synthetic-demo output.

14.2 System Results

- 31 mounted active endpoints are visible across my final backend routers: AI, progress, courses, instructor, goals, leaderboard, history, and notifications.
- 2 additional dataset endpoints exist in backend/routers/dataset.py but are not mounted in the final backend app.
- 6 courses and 41 lessons are provided by the course router.
- 12 research datasets are presented in the dataset library UI.
- The instructor router returns platform stats such as 247 learners, 38 active today, and 76.4 percent average platform accuracy as demo data.

15. Deployment

The project is prepared for local development and demo deployment. The backend can be run with Uvicorn from the backend folder, while the frontend and portfolio use Vite. The branch history includes Dockerfiles and docker-compose.yml in the project repository. The final docs include a deployment guide, but no verified cloud deployment URL was found in the local materials.

```
Backend:
cd backend
pip install -r requirements.txt
python -m uvicorn main:app --reload --port 8000
```

```
ML training:
cd ml
pip install -r requirements_ml.txt
python train_classifier.py
```

```
Frontend:
```

```
cd frontend
npm install
npm run dev
```

16. Challenges and Limitations

- Large datasets such as WLASL and RWTH-PHOENIX are too large to store directly in Git and require local setup.
- The final dataset_pipeline.py has a syntax error in generate_dataset_summary_report and should be corrected before execution.
- Most final backend routers use hardcoded or generated demo data rather than database persistence.
- The final backend uses a rule-based evaluator while the stronger Week 2 branch includes a trained-model service wrapper.
- The dataset router exists but is not mounted in backend/main.py.
- Auth/profile files exist, but the active final backend does not mount all auth/profile routers.
- The latest chinmayee_milestone3 branch removes a Milestone4 subfolder, while the final assembled project still contains the Milestone 4-style files.

17. Future Scope

- Fix dataset_pipeline.py and add automated smoke tests for dataset metadata and preprocessing helpers.
- Mount the dataset router in backend/main.py if the Dataset Library should read from the backend.
- Connect routers to PostgreSQL models for real persistence of practice history, mastery, enrollments, goals, and notifications.
- Load the trained joblib classifier in the final backend instead of relying only on geometric rules.
- Train and evaluate on real extracted landmarks from ASL Alphabet, WLASL, ASL-Citizen, ASLLVD, or other documented datasets.
- Add automated API tests for validation errors, happy paths, and unavailable model behavior.
- Tighten production security by restricting CORS, using robust password hashing, and enforcing role-based authorization on protected routes.

18. Conclusion

My work forms a major backend, AI, and dataset foundation for SignLearn AI. Across the reviewed branches, I contributed dataset research and setup rules, a working AI gesture evaluation endpoint, landmark feature extraction, model training support, dataset metadata handling, a dataset library UI, many learning-platform backend routers, milestone documentation, and a portfolio artifact. The result is a project that can explain its data sources, accept sign-practice inputs, return coaching-style feedback, and support the surrounding learning experience through structured APIs.

19. References

- Final Document Overview.pdf - template structure and visual reference.
- CHINMAYEE_WORK_DETAILS.md and MEMBERS_WORK/02_Chinmayee_Badiger/MY_WORK_README.md.

- chinmayee-week1 branch - Dataset_Integration_Guide.md and dataset processing explanation.
- chinmayee-week-2 branch - Milestone2_Task2_Explanation.md, AI endpoint, ML service files, and saved model artifact.
- chinmayee-week2-milestone2 and chinmayee_milestone3 branches - dataset UI, milestone archive, portfolio, and branch cleanup.
- Final project backend, frontend, ml, docs, MEMBERS_WORK, and team_master folders.
- Project dataset references: Sign Language MNIST, ASL Alphabet, WLASL, RWTH-PHOENIX, ASLLVD, OpenASL, ASL-Citizen, How2Sign, MS-ASL, AUTSL, NCSLGR, SignBD, CSL-Daily, and INCLUDE.